

## SQL Join Types

In SQL, joins are used to combine rows from two or more tables based on a related column between them. Here are the main types of joins:

**1. INNER JOIN:** Returns records that have matching values in both tables.

```
```sql  
  
SELECT columns  
  
FROM table1  
  
INNER JOIN table2  
  
ON table1.column = table2.column;  
```
```

**2. LEFT (OUTER) JOIN:** Returns all records from the left table (table1), and the matched records from the right table (table2). If there is no match, the result is NULL on the side of the right table.

```
```sql  
  
SELECT columns  
  
FROM table1  
  
LEFT JOIN table2  
  
ON table1.column = table2.column;  
```
```

**3. RIGHT (OUTER) JOIN:** Returns all records from the right table (table2), and the matched records from the left table (table1). If there is no match, the result is NULL on the side of the left table.

```
```sql
```

```
SELECT columns  
FROM table1  
RIGHT JOIN table2  
ON table1.column = table2.column;  
...
```

**4. FULL (OUTER) JOIN:** Returns all records when there is a match in either left (table1) or right (table2) table. Records that do not match are also included, with NULLs in place of missing data.

```
```sql  
SELECT columns  
FROM table1  
FULL OUTER JOIN table2  
ON table1.column = table2.column;  
...
```

**5. CROSS JOIN:** Returns the Cartesian product of the two tables, meaning it combines each row of the first table with each row of the second table.

```
```sql  
SELECT columns  
FROM table1  
CROSS JOIN table2;  
...
```

**6. SELF JOIN:** A regular join, but the table is joined with itself. It's useful for querying hierarchical data or comparing rows within the same table.

```
```sql
SELECT a.columns, b.columns
FROM table1 a, table1 b
WHERE condition;
```
```

**7. NATURAL JOIN:** A type of join that automatically joins tables based on columns with the same names and compatible types. It can be an INNER JOIN, LEFT JOIN, or RIGHT JOIN.

```
```sql
SELECT columns
FROM table1
NATURAL JOIN table2;
WHERE condition;(Optional)
```
```

Each type of join serves a different purpose and can be used depending on the specific requirements of the data retrieval operation.

**Outer Join:** An **Outer Join** returns all matching records from two tables **plus** the unmatched records from one or both tables.

## **View**

A **view** in SQL is a **virtual table** that is defined by a SQL query. Unlike a real table, a view does not store data physically. Instead, it provides a dynamic result set based on the query it encapsulates.

```

```Sql
CREATE VIEW view_name AS
SELECT column1, column2, ...
FROM table_name
WHERE condition;
...

```

- **view\_name:** The name of the view you want to create.
- **SELECT statement:** Defines the columns and rows included in the view. It can include filters, joins, aggregations, and other SQL features.

### For Updating view

```

```Sql
UPDATE view_name
SET column1 = value1, column2 = value2, ...
WHERE condition;
.....

```

## Integrity

**Integrity** in databases refers to the accuracy, consistency, and reliability of data. It ensures that the data remains correct, valid, and secure throughout its lifecycle, even during updates, deletions, or transactions.

There are different types of integrity, such as:

- **Entity Integrity:** Ensures each record in a table is unique, typically using a primary key.
- **Referential Integrity:** Ensures relationships between tables remain consistent, often using foreign keys.
- **Domain Integrity:** Ensures data values fall within a valid range or set.

## Features of Good Relational Designs

A **good relational design** ensures data integrity, efficiency, and ease of use in a database. The key features are:

1. **Normalization** – Eliminates redundancy and ensures data consistency by organizing data into well-structured tables (e.g., 1NF, 2NF, 3NF).
2. **Proper Use of Keys** – Uses **Primary Keys** for uniqueness and **Foreign Keys** for relationships.
3. **Minimal Redundancy** – Avoids duplicate data, reducing storage needs and update anomalies.
4. **Data Integrity** – Enforces constraints like **NOT NULL**, **UNIQUE**, **CHECK**, and **REFERENTIAL INTEGRITY** (foreign key constraints).
5. **Scalability** – Supports growth without performance issues.
6. **Efficient Indexing** – Uses indexes to speed up queries while balancing storage and performance.
7. **Clear Relationships** – Uses **One-to-One**, **One-to-Many**, and **Many-to-Many** relationships correctly.
8. **Atomicity** – Stores data in the smallest meaningful units to avoid partial updates.
9. **Consistency & Accuracy** – Ensures that related data remains synchronized across tables.
10. **Security & Access Control** – Implements roles, permissions, and authentication to protect data.

## Decomposition in DBMS: Lossless vs. Lossy

Decomposition in DBMS is the process of breaking a relation (table) into two or more smaller relations to reduce redundancy and maintain consistency.

### 1. Lossless Decomposition (Preferred) ✓

- No data loss when decomposed tables are joined back.
- Ensures that the original table can be fully reconstructed.
- Used in Normalization to remove redundancy without losing information.

#### ✓ Example (Lossless Decomposition)

Original Table (R):

| Student_ID | Name | Course  | Instructor |
|------------|------|---------|------------|
| 101        | Alex | Math    | John       |
| 102        | Bob  | Science | Mary       |

Decomposed into two tables:

R1 (Student\_Course)

| Student_ID | Course  |
|------------|---------|
| 101        | Math    |
| 102        | Science |

R2 (Course\_Instructor)

| Course  | Instructor |
|---------|------------|
| Math    | John       |
| Science | Mary       |

When we JOIN R1 and R2, we get back the original table ✓ (Lossless).

### 2. Lossy Decomposition (Not Recommended) ✗

- **Some data is lost** when decomposed tables are joined back.
- The original relation **cannot be fully reconstructed**.
- Usually happens when functional dependencies are not preserved

### ✗ Example (Lossy Decomposition)

Original Table (R):

| Student_ID | Name | Course  | Instructor |
|------------|------|---------|------------|
| 101        | Alex | Math    | John       |
| 102        | Bob  | Science | Mary       |

Wrongly decomposed into:

R1 (Student\_Name)

| Student_ID | Name |
|------------|------|
| 101        | Alex |
| 102        | Bob  |

R2 (Course\_Instructor)

| Course  | Instructor |
|---------|------------|
| Math    | John       |
| Science | Mary       |

If we try to JOIN R1 and R2, we cannot restore the Course column correctly ✗.

## Functional Dependencies

### Slide part-----

#### Functional Dependencies in DBMS – Expanded Concepts

Functional dependencies (FDs) help define rules and constraints within a database to maintain data integrity and consistency. They play a crucial role in database normalization and ensure that attributes depend on keys in a meaningful way.

---

#### 1. Understanding Functional Dependencies

A **functional dependency** exists when a set of attributes (X) uniquely determines another set of attributes (Y).

- **Notation:**  $X \rightarrow Y$
- **Meaning:** If two rows have the same value for X, they must have the same value for Y.

### Example

| StudentID | Name  | Department |
|-----------|-------|------------|
| 101       | Alice | CSE        |
| 102       | Bob   | ECE        |
| 103       | Alice | CSE        |

Here, **StudentID**  $\rightarrow$  **Name** holds because each StudentID corresponds to a unique Name.

---

## 2. Real-World Constraints and Legal Instances

In real-world applications, databases must satisfy specific rules. These rules define **legal instances** of relations.

### Examples of Constraints in a University Database

1. **Students and instructors have unique IDs**  $\rightarrow$  ID  $\rightarrow$  NameID
2. **Each student/instructor belongs to one department**  $\rightarrow$  ID  $\rightarrow$  DeptName
3. **Each department has a single budget and building**  $\rightarrow$   
DeptName  $\rightarrow$  Budget, Building

A **legal instance** of a relation is one where all real-world constraints hold.

---

## 3. Formal Definition of Functional Dependency

Let **R** be a relation schema.

- If  $\alpha \subseteq R$  and  $\beta \subseteq R$ , the functional dependency  $\alpha \rightarrow \beta$  holds if, for every legal relation  $r(R)$ , whenever two tuples have the same value for  $\alpha$ , they also have the same value for  $\beta$ .
- **Mathematical Notation:**
  - If  $t_1[\alpha] = t_2[\alpha]$ , then  $t_1[\beta] = t_2[\beta]$

### Example

Relation:  $r(A, B)$

| A | B |
|---|---|
| 1 | 5 |
| 3 | 7 |

- Here,  $B \rightarrow A$  holds (same B value means same A value).
  - However,  $A \rightarrow B$  does NOT hold (same A value does not imply same B value).
- 

### 4. Closure of Functional Dependencies ( $F^+$ )

The **closure** of a set of functional dependencies is the set of all functional dependencies that can be **logically inferred** from a given set  $F$ .

### Example

Given FDs:

- $A \rightarrow B$
- $B \rightarrow C$

Then, we can infer:

- $A \rightarrow C$

The complete set of inferred dependencies forms  $F^+$  (**closure of  $F$** ).

---

### 5. Keys and Functional Dependencies

- A **superkey (K)** for relation schema  $R$  satisfies:  
 $K \rightarrow R$  (K uniquely determines all attributes of R).
- A **candidate key** (minimal superkey) is a superkey where **no subset** of K determines R.

- **Example:**
    - In **Employee (ID, Name, Salary, DeptName, Building, Budget)**:
      - **ID  $\rightarrow$  Name, Salary, DeptName, Building, Budget** (ID is a superkey).
      - **DeptName  $\rightarrow$  Building** (DeptName does not determine all attributes, so it is not a key).
- 

## 6. Use of Functional Dependencies

- **Checking if a relation is legal** under a given set of FDs.
- **Enforcing constraints** to maintain database integrity.
- **Testing relations** against FDs to avoid anomalies.

*Note:* A specific instance may satisfy an FD by chance, but that does not mean the FD is universally valid.

### Example

A relation may **incidentally** satisfy **Name  $\rightarrow$  ID**, but this does not hold in general because multiple students can have the same name.

---

## 7. Trivial Functional Dependencies

A **trivial FD** is one where the right-hand side is a subset of the left-hand side.

### Examples

- **ID, Name  $\rightarrow$  ID, Name** (trivial because ID is part of {ID, Name}).
- **Name  $\rightarrow$  Name** (trivial because Name determines itself).

### General Rule:

**$\alpha \rightarrow \beta$  is trivial if  $\beta \subseteq \alpha$ .**

---

## Functional Dependencies

**Functional Dependency (FD)** is a relationship between attributes in a table, where one attribute uniquely determines another. It helps in **Normalization** by identifying dependencies and eliminating redundancy.

### Definition

A functional dependency  $A \rightarrow B$  means:

- If two rows have the same value for **A**, they must also have the same value for **B**.
- **A is the determinant**, and **B is the dependent**.

✓ Example:

| Student_ID | Name | Course  |
|------------|------|---------|
| 101        | Alex | Math    |
| 102        | Bob  | Science |
| 101        | Alex | Science |

Here,  $Student\_ID \rightarrow Name$  because for the same  $Student\_ID$  (101), the  $Name$  (Alex) remains the same.

## Finding the Closure of F ( $F^+$ )

The closure  $F^+$  is found by repeatedly applying Armstrong's Axioms to infer all possible functional dependencies.

### Example 1:

#### Given Functional Dependencies (F)

$F = \{ A \rightarrow B, B \rightarrow C \}$

#### Finding $F^+$

1. **Given FDs:**
  - $A \rightarrow B$
  - $B \rightarrow C$
2. **Using Transitivity** ( $A \rightarrow B$  and  $B \rightarrow C \Rightarrow A \rightarrow C$ )
  - $A \rightarrow C$

### 3. Final Closure ( $F^+$ )

$$F^+ = \{ A \rightarrow B, B \rightarrow C, A \rightarrow C \}$$

## 2 Types of Functional Dependencies

### 1. Trivial Dependency

If  $B$  is a subset of  $A$ , then  $A \rightarrow B$  is trivial.

✓ Example:

| Student_ID | Name |
|------------|------|
| 101        | Alex |

Here,  $(\text{Student\_ID}, \text{Name}) \rightarrow \text{Name}$  is trivial because "Name" is already part of the left side.

### 2. Non-Trivial Dependency

If  $B$  is NOT a subset of  $A$ , then  $A \rightarrow B$  is non-trivial.

✓ Example:

$\text{Student\_ID} \rightarrow \text{Name}$  (Name is not part of Student\_ID),  
(↓)

### 3. Partial Dependency (Violates 2NF ❌)

When a **non-key attribute** depends only on **part** of the **composite primary key** (not the whole key).

✅ Example:

| Student_ID | Course  | Instructor |
|------------|---------|------------|
| 101        | Math    | John       |
| 102        | Science | Mary       |

Here, (Student\_ID, Course) is the **Primary Key**, but **Instructor** depends only on **Course**, not the full key.

❌ Fix: Decompose into separate tables.

### 4. Transitive Dependency (Violates 3NF ❌)

If  $A \rightarrow B$  and  $B \rightarrow C$ , then  $A \rightarrow C$  (indirect dependency).

✅ Example:

| Student_ID | Dept_ID | Dept_Name |
|------------|---------|-----------|
| 101        | 10      | Science   |

Here,  $Student\_ID \rightarrow Dept\_ID$  and  $Dept\_ID \rightarrow Dept\_Name$ , so  $Student\_ID \rightarrow Dept\_Name$  is a **transitive dependency**.

❌ Fix: Create a separate **Department** table.

## 3 Role in Normalization

| FD Type               | Normalization Impact |
|-----------------------|----------------------|
| Partial Dependency    | Removed in 2NF       |
| Transitive Dependency | Removed in 3NF       |

## Uses of Functional Dependencies (FDs)

1. **Finding Candidate Keys** – Helps identify minimal keys that uniquely define records.
2. **Normalization** – Reduces redundancy and ensures efficient database design (1NF, 2NF, 3NF, BCNF).
3. **Removing Redundant Data** – Helps split tables to avoid unnecessary duplication.
4. **Checking Lossless Decomposition** – Ensures no data is lost when breaking tables.

5. **Ensuring Data Integrity** – Prevents inconsistent updates by maintaining relationships.
6. **Query Optimization** – Helps the database optimize queries by reducing computations.

## Functional Dependency & Its Types (4 Marks)

### Definition of Functional Dependency:

A **functional dependency (FD)** in a relational database occurs when one attribute uniquely determines another.

If  $X \rightarrow Y$ , then knowing  $X$  means we can determine  $Y$ .

♦ **Example:** In a **Student** table:

- $\text{RollNo} \rightarrow \text{FirstName}$  (RollNo uniquely determines FirstName)

### Types of Functional Dependencies:

#### 1. Trivial Functional Dependency:

- If  $Y \subseteq X$ , then  $X \rightarrow Y$  is trivial.
- Example:  $\{\text{RollNo}, \text{FirstName}\} \rightarrow \text{RollNo}$  (RollNo is already in the left side).

#### 2. Non-Trivial Functional Dependency:

- If  $Y$  is **not a subset of  $X$** , then  $X \rightarrow Y$  is non-trivial.
- Example:  $\text{RollNo} \rightarrow \text{FirstName}$  (FirstName is not part of RollNo).

#### 3. Partial Functional Dependency:

- When **only a part** of a composite key determines another attribute.
- Example: In a relation (**Student\_ID, Course\_ID, Student\_Name, Marks**),  
 $(\text{StudentID}, \text{CourseID}) \rightarrow \text{Marks}$

But if **Student\_Name depends only on Student\_ID**, then it's a **partial dependency**

#### 4. Transitive Functional Dependency:

- If  $X \rightarrow Y$  and  $Y \rightarrow Z$ , then  $X \rightarrow Z$ .
- Example:  $\text{RollNo} \rightarrow \text{Class}$  and  $\text{Class} \rightarrow \text{ClassTeacher} \Rightarrow \text{RollNo} \rightarrow \text{ClassTeacher}$ .

#### 5. Multivalued Dependency (MVD):

- If  $X \twoheadrightarrow Y$ , changing  $Y$ 's value does not affect other attributes.
- Example:  $\text{StudentID} \twoheadrightarrow \text{Hobby}$ , where a student may have multiple hobbies.

**Normalization** is a database design process that organizes data to reduce **redundancy** and improve **data integrity**. It involves dividing large tables into smaller ones and defining relationships between them.

#### Types of Keys:

| Key Type      | Definition                                                               | Example (Student Table: Student_ID, Name, Course)                 |
|---------------|--------------------------------------------------------------------------|-------------------------------------------------------------------|
| Super Key     | A set of attributes that uniquely identifies a row.                      | {Student_ID}, {Student_ID, Name}                                  |
| Candidate Key | The minimal super key (no extra attributes).                             | {Student_ID}                                                      |
| Primary Key   | The chosen candidate key that uniquely identifies records.               | Student_ID                                                        |
| Foreign Key   | An attribute in one table that refers to a primary key in another table. | Course_ID (references Course table)                               |
| Composite Key | A key made of multiple attributes.                                       | {Student_ID, Course}                                              |
| Alternate Key | Candidate keys not chosen as the primary key.                            | If {Student_ID} is primary, then {Name} (if unique) is alternate. |
| Unique Key    | Ensures unique values but allows NULLs.                                  | Email (each student must have a unique email)                     |

## Decomposition

Decomposition is the process of breaking a relation (table) into two or more smaller relations to eliminate redundancy and ensure data integrity. It is mainly used in database normalization to remove anomalies (insertion, update, and deletion anomalies)

### lossy decomposition

A **lossy decomposition** occurs when a relation is decomposed into smaller relations, and upon rejoining these smaller relations, some information is lost, meaning the original relation cannot be fully reconstructed.

### Example:

Consider a relation **Employee**(Emp\_ID, Emp\_Name, Dept, Salary) with the following data:

| Emp_ID | Emp_Name | Dept | Salary |
|--------|----------|------|--------|
| 1      | John     | HR   | 50000  |
| 2      | Alice    | IT   | 60000  |
| 3      | Bob      | HR   | 55000  |

Decompose the relation into two smaller relations:

1. **Emp1**(Emp\_ID, Emp\_Name, Dept)
2. **Emp2**(Dept, Salary)

Rejoining these relations on **Dept** gives:

| Emp_ID | Emp_Name | Dept | Salary |
|--------|----------|------|--------|
| 1      | John     | HR   | 50000  |
| 3      | Bob      | HR   | 50000  |
| 2      | Alice    | IT   | 60000  |

Here, the **Emp\_ID** is lost, and the rejoined relation is incorrect because it doesn't capture the original connection between **Emp\_ID** and **Salary**. Hence, this is a **lossy decomposition**.

## lossless decomposition

A **lossless decomposition** occurs when a relation is decomposed into smaller relations, and upon rejoining these relations, the original relation can be **perfectly reconstructed** without any loss of information.

### Example:

Consider a relation **Employee**(Emp\_ID, Emp\_Name, Dept, Salary) with the following data:

| Emp_ID | Emp_Name | Dept | Salary |
|--------|----------|------|--------|
| 1      | John     | HR   | 50000  |
| 2      | Alice    | IT   | 60000  |
| 3      | Bob      | HR   | 55000  |

Decompose the relation into two smaller relations:

1. **Emp1**(Emp\_ID, Emp\_Name, Dept)
2. **Emp2**(Emp\_ID, Salary)

When we rejoin these relations on **Emp\_ID**, we get:

| Emp_ID | Emp_Name | Dept | Salary |
|--------|----------|------|--------|
| 1      | John     | HR   | 50000  |
| 2      | Alice    | IT   | 60000  |
| 3      | Bob      | HR   | 55000  |

The original relation can be **accurately reconstructed** by rejoining, so this is a **lossless decomposition**.

**\*\*** A decomposition of R into  $R_1$  and  $R_2$  is lossless decomposition

if at least one of the following dependencies is in  $F^+$ :

1.  $(R_1 \cap R_2) \rightarrow R_1$

2.  $(R_1 \cap R_2) \rightarrow R_2$

### **Example: Functional Dependencies and Lossless Decomposition**

**Given:**

- **Relation:**  $R(A, B, C)$
  - **Functional Dependencies:**  $F = \{ A \rightarrow B, B \rightarrow C \}$
- 

#### **Case 1: Decomposing R into $(R_1 = (A, B), R_2 = (B, C))$**

- **Intersection:**  $R_1 \cap R_2 = \{B\}$
  - **Dependency Check:** Since  $B \rightarrow BC$  (B determines both B and C), the decomposition is **lossless**.
- 

#### **Case 2: Decomposing R into $(R_1 = (A, B), R_2 = (A, C))$**

- **Intersection:**  $R_1 \cap R_2 = \{A\}$
  - **Dependency Check:** Since  $A \rightarrow AB$  (A determines A and B), the decomposition is **lossless**.
- 

## **Dependency Preservation in DBMS**

### **What is Dependency Preservation?**

- Dependency Preservation ensures that all functional dependencies (FDs) from the original table are maintained after decomposition.
- It helps in Normalization, ensuring data consistency without losing important relationships.

- If dependencies are not preserved, extra joins are needed, which can reduce efficiency.
- 

## Why is Dependency Preservation Important?

- ✓ Ensures all functional dependencies remain in decomposed tables.
- ✓ Reduces complex joins when querying data.
- ✓ Maintains data consistency without unnecessary redundancy.

## How to Check Dependency Preservation?

- Suppose a relation R is decomposed into R1, R2, R3, ...
- If all functional dependencies (FDs) from R can be enforced in some decomposed table without extra joins, then the decomposition is dependency-preserving.

## Dependency Preservation Example Using Student, Course, and Instructor

### Given Data:

Let the original relation be:

**R(Student\_ID, Student\_Name, Course\_Name, Instructor\_Name, Instructor\_Department)**

### Functional Dependencies (F):

1. **Student\_ID → Student\_Name**  
(Student ID determines Student Name)
  2. **Course\_Name → Instructor\_Name, Instructor\_Department**  
(Course Name determines Instructor Name and Instructor Department)
  3. **Instructor\_Name → Instructor\_Department**  
(Instructor Name determines Instructor Department)
-

### Decomposing Relation R:

We decompose **R** into the following relations:

1. **R1(Student\_ID, Student\_Name, Course\_Name)**
  2. **R2(Course\_Name, Instructor\_Name, Instructor\_Department)**
- 

### Checking for Dependency Preservation:

- **FD 1 (Student\_ID  $\rightarrow$  Student\_Name):**  
This dependency is preserved in **R1**, as **Student\_ID** uniquely determines **Student\_Name**.
  - **FD 2 (Course\_Name  $\rightarrow$  Instructor\_Name, Instructor\_Department):**  
This dependency is preserved in **R2**, as **Course\_Name** determines both **Instructor\_Name** and **Instructor\_Department**.
  - **FD 3 (Instructor\_Name  $\rightarrow$  Instructor\_Department):**  
This dependency is preserved in **R2**, as **Instructor\_Name** determines **Instructor\_Department**.
-

# Normalization in Databases

## What is Normalization?

Normalization is a process in database design used to reduce redundancy and improve data integrity by organizing data into multiple related tables. It helps prevent anomalies during data insertion, deletion, and updating.

---

## Why is Normalization Needed?

1. **Eliminates Redundancy** → Avoids storing duplicate data.
2. **Ensures Data Integrity** → Prevents inconsistent data entries.
3. **Reduces Anomalies** → Helps in avoiding insertion, update, and deletion issues.
4. **Efficient Storage** → Saves disk space by minimizing unnecessary data.
5. **Improves Query Performance** → Speeds up queries by reducing the size of data sets.

## Types (Forms) of Normalization

Normalization is divided into different normal forms (NF), each building upon the previous one.

### 1. First Normal Form (1NF) - Atomicity

#### Condition:

- Each column should have atomic (indivisible) values.
- Each column must have unique values (no duplicate columns).
- The table must have a primary key.

Example (Not in 1NF):

| Student_ID | Name  | Courses   |
|------------|-------|-----------|
| 101        | John  | Math, CS  |
| 102        | Alice | Physics   |
| 103        | Bob   | Math, Bio |

Problem:

- "Courses" column has multiple values, which violates atomicity.

Solution (1NF Applied):

| Student_ID | Name  | Course  |
|------------|-------|---------|
| 101        | John  | Math    |
| 101        | John  | CS      |
| 102        | Alice | Physics |
| 103        | Bob   | Math    |
| 103        | Bob   | Bio     |

## 2. Second Normal Form (2NF) - Remove Partial Dependency

Condition:

- Table must be in **1NF**.
- Every non-key column must be fully functionally dependent on the **entire primary key** (no **partial dependencies**).

#### Example (Not in 2NF)

| Student_ID | Course  | Instructor | Instructor_Age |
|------------|---------|------------|----------------|
| 101        | Math    | Smith      | 45             |
| 101        | CS      | Johnson    | 40             |
| 102        | Physics | Brown      | 50             |

#### Problem:

- Instructor and Instructor\_Age depend only on Course, **not on Student\_ID**.
- This causes data redundancy if the same instructor teaches multiple courses.

#### Solution (2NF Applied - Split into two tables)

##### Student-Course Table

| Student_ID | Course  |
|------------|---------|
| 101        | Math    |
| 101        | CS      |
| 102        | Physics |

##### Course-Instructor Table

| Course  | Instructor | Instructor_Age |
|---------|------------|----------------|
| Math    | Smith      | 45             |
| CS      | Johnson    | 40             |
| Physics | Brown      | 50             |

Now, instructor details are stored separately without redundancy.

---

### 3. Third Normal Form (3NF) - Remove Transitive Dependency

#### Condition:

- Table must be in **2NF**.
- There should be **no transitive dependency** (a column should depend only on the primary key and **not on another non-key column**).

#### Example (Not in 3NF)

| Student_ID | Student_Name | City | Zip_Code | State |
|------------|--------------|------|----------|-------|
| 101        | John         | NY   | 10001    | NY    |
| 102        | Alice        | LA   | 90001    | CA    |

#### Problem:

- City and State depend on Zip\_Code, not on Student\_ID (this is a transitive dependency).

#### Solution (3NF Applied - Split into two tables)

##### Student Table

| Student_ID | Student_Name | Zip_Code |
|------------|--------------|----------|
| 101        | John         | 10001    |
| 102        | Alice        | 90001    |

##### Zip Code Table

| Zip_Code | City | State |
|----------|------|-------|
| 10001    | NY   | NY    |
| 90001    | LA   | CA    |

Now, City and State are correctly linked to Zip\_Code, avoiding redundancy.

## 4. Boyce-Codd Normal Form (BCNF) - Advanced 3NF

#### Condition:

- Table must be in **3NF**.
- Every determinant must be a **candidate key**.

### Example (Not in BCNF)

| Professor | Department | Course-Taught |
|-----------|------------|---------------|
| Smith     | CS         | DBMS          |
| Smith     | CS         | OOP           |
| Brown     | Math       | Calculus      |

#### Problem:

- "Professor" decides "Department," but "Course-Taught" also partially depends on "Professor."
- This can cause redundancy if a professor teaches multiple courses.

#### Solution (BCNF Applied - Split into two tables)

##### Professor Table

| Professor | Department |
|-----------|------------|
| Smith     | CS         |
| Brown     | Math       |

##### Course Table

| Course-Taught | Professor |
|---------------|-----------|
| DBMS          | Smith     |
| OOP           | Smith     |
| Calculus      | Brown     |

## 4th Normal Form (4NF) – Remove Multi-Valued Dependency

### Condition:

- The table must already be in **Boyce-Codd Normal Form (BCNF)**.
- **No multi-valued dependencies (MVDs)** should exist (i.e., one column should not determine multiple independent values of another column).

### Example (Not in 4NF - Table with Multi-Valued Dependency)

| Student_ID | Phone_Number | Course  |
|------------|--------------|---------|
| 101        | 1234567890   | Math    |
| 101        | 1234567890   | Physics |
| 101        | 9876543210   | Math    |
| 101        | 9876543210   | Physics |

#### Problem:

- $\text{Student\_ID} \twoheadrightarrow \text{Phone\_Number}$  (A student has multiple phone numbers).
- $\text{Student\_ID} \twoheadrightarrow \text{Course}$  (A student enrolls in multiple courses).
- **Phone\_Number and Course are independent**, so storing them in the same table causes redundancy.

### Solution (4NF Applied - Split into Two Tables)

#### Student-Phone Table

| Student_ID | Phone_Number |
|------------|--------------|
| 101        | 1234567890   |
| 101        | 9876543210   |

#### Student-Course Table

| Student_ID | Course  |
|------------|---------|
| 101        | Math    |
| 101        | Physics |

## Transaction

### Transaction in DBMS

A **transaction** in a **Database Management System (DBMS)** is a sequence of operations performed as a single logical unit of work. A transaction must be **atomic**, meaning either all of its operations are executed successfully or none at all.

## Properties of Transactions (ACID Properties)

1. **Atomicity** – Ensures that a transaction is **all or nothing** (no partial execution).
2. **Consistency** – Ensures the database remains **valid** before and after the transaction.
3. **Isolation** – Ensures that transactions do not **interfere** with each other.
4. **Durability** – Ensures that once a transaction is **committed**, it remains permanent.

## States of a Transaction

1. **Active** – Transaction is in progress.
2. **Partially Committed** – All operations are executed, but not yet saved.
3. **Committed** – Changes are permanently saved to the database.
4. **Failed** – An error occurred, leading to rollback.
5. **Aborted** – Transaction is rolled back, returning the database to its previous state.

## Transaction Control Commands

1. **COMMIT** – Saves all changes made by the transaction.
2. **ROLLBACK** – Undoes all changes made by the transaction.
3. **SAVEPOINT** – Creates a checkpoint to roll back to a specific point.
4. **SET TRANSACTION** – Defines the properties of a new transaction.

## The two main operations in a DBMS transaction are:

### 1. Read (Retrieve) Operation

- Fetches data from the database without modifying it.
- Example in SQL:

```
SELECT * FROM Customers WHERE CustomerID = 1;
```

### 2. Write (Modify) Operation

- Inserts, updates, or deletes data in the database.
- Example in SQL:

UPDATE Customers SET Name = 'John Doe' WHERE CustomerID = 1;

## Schedule

A **schedule** in **DBMS** is an **ordered sequence of transactions** that defines how multiple transactions are executed concurrently while maintaining consistency. It ensures that transactions follow the ACID properties, especially **isolation**.

---

### Types of Schedules

#### 1. Serial Schedule

- Transactions execute **one after another** without overlapping.
- Ensures consistency but reduces concurrency.
- Example:

T1: Read(X) → Write(X) → Read(Y) → Write(Y)

T2: Read(A) → Write(A) → Read(B) → Write(B)

#### 2. Non-Serial Schedule

- Transactions **overlap**, meaning operations of different transactions are interleaved.
- Requires concurrency control to maintain consistency.

#### 3. Serializable Schedule

- A **non-serial schedule** that **produces the same result** as a serial schedule.
- Ensures correctness while allowing concurrency.
- Can be checked using **Conflict Serializability** or **View Serializability**.

#### 4. Conflict Serializable Schedule

- A schedule where transactions **can be rearranged** into a **serial schedule** by swapping **non-conflicting** operations.

#### 5. View Serializable Schedule

- A schedule that produces the **same final output** as a serial schedule, but may not be conflict serializable.

## 6. Cascadeless Schedule

- A schedule that **prevents cascading rollbacks** by ensuring that transactions **do not read uncommitted data** from other transactions.

## 7. Recoverable Schedule

- A schedule where a transaction **commits only after all transactions it depends on have committed**.

---

Would you like an example or a simple diagram for better understanding? 🚀